

Министерство образования Республики Беларусь  
Учреждение образования «Белорусский государственный университет  
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей  
Кафедра информатики  
Дисциплина «Объектно-ориентированное программирование»

«К защите допустить»  
Руководитель курсового проекта  
ассистент кафедры информатики  
\_\_\_\_\_ А.Г. Бокун  
\_\_\_\_.\_\_\_\_.2026

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА**  
к курсовому проекту  
на тему:  
**«ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ ГЕНЕРАЦИИ ИЗОБРАЖЕНИЙ  
ФРАКТАЛЬНОГО ПЛАМЕНИ»**

БГУИР КП 6-05 0612 02 26 ПЗ

Выполнил студент группы 453501  
СКРОБОТ Денис Алексеевич

\_\_\_\_\_  
(подпись студента)

Курсовой проект представлен на  
проверку \_\_\_\_\_.\_\_\_\_\_.2026

\_\_\_\_\_  
(подпись студента)

Минск 2026

# СОДЕРЖАНИЕ

|                                                                      |    |
|----------------------------------------------------------------------|----|
| Введение . . . . .                                                   | 4  |
| 1 Анализ аналогов . . . . .                                          | 5  |
| 1.1 Apophysis . . . . .                                              | 5  |
| 1.2 JWildfire . . . . .                                              | 6  |
| 1.3 Chaotica . . . . .                                               | 7  |
| 1.4 Сравнительный анализ . . . . .                                   | 8  |
| 2 Формирование функциональных требований . . . . .                   | 8  |
| 3 Анализ литературы . . . . .                                        | 9  |
| 3.1 Принципы объектно-ориентированного проектирования . . . . .      | 9  |
| 3.2 Принципы SOLID и качество кода . . . . .                         | 10 |
| 3.3 Шаблоны проектирования . . . . .                                 | 10 |
| 3.4 Моделирование предметной области . . . . .                       | 10 |
| 3.5 Слоистая организация приложения . . . . .                        | 11 |
| 3.6 Контракты и переносимость моделей . . . . .                      | 11 |
| 4 Проектирование системы . . . . .                                   | 12 |
| 4.1 Общая архитектура . . . . .                                      | 12 |
| 4.2 Frontend . . . . .                                               | 13 |
| 4.3 API gateway . . . . .                                            | 14 |
| 4.4 IAM . . . . .                                                    | 14 |
| 4.5 Generator . . . . .                                              | 15 |
| 4.6 Процесс генерации фрактального пламени . . . . .                 | 17 |
| 4.7 Передача прогресса генерации . . . . .                           | 18 |
| 4.8 Развёртывание . . . . .                                          | 18 |
| 4.9 Технологии используемые для решения поставленных задач . . . . . | 20 |
| 4.9.1 Язык программирования Java . . . . .                           | 20 |
| 4.9.2 Фреймворк Spring . . . . .                                     | 21 |
| 4.9.3 Язык Go . . . . .                                              | 22 |
| 4.9.4 Frontend: TypeScript, Vue, Tailwind CSS и Axios . . . . .      | 22 |
| 4.9.5 Система сборки Gradle . . . . .                                | 23 |
| 4.9.6 База данных PostgreSQL . . . . .                               | 23 |
| 4.9.7 MinIO и S3-совместимое хранилище . . . . .                     | 23 |
| 4.9.8 gRPC . . . . .                                                 | 24 |
| 4.9.9 JWT . . . . .                                                  | 24 |
| 4.9.10 Контейнеризация Docker и Docker Compose . . . . .             | 24 |
| 4.9.11 Среда разработки Visual Studio Code . . . . .                 | 25 |
| 5 Тестирование программного обеспечения . . . . .                    | 26 |
| 5.1 Стратегия тестирования . . . . .                                 | 26 |
| 5.2 Сценарии тестирования . . . . .                                  | 26 |
| Заключение . . . . .                                                 | 28 |
| Список использованных источников . . . . .                           | 30 |

## ВВЕДЕНИЕ

Фрактальная графика относится к области визуализации сложных математических моделей и активно используется в цифровом искусстве, дизайне и научной визуализации. Одним из наиболее выразительных направлений является генерация фрактального «пламени», предложенная С. Дрейвсом, где изображение формируется итеративными нелинейными преобразованиями и последующей тональной обработкой плотности [1]. Благодаря богатству форм и высокой детализации такие изображения востребованы как в творческих проектах, так и в обучении математическому моделированию.

Современные инструменты генерации фракталов чаще представлены настольными приложениями и ориентированы на локальную работу пользователя. При этом растёт спрос на веб-сервисы, обеспечивающие доступ с разных устройств, хранение истории и возможность публикации результатов. Для решения этих задач требуется система, сочетающая удобный пользовательский интерфейс, вычислительно ёмкий рендеринг и надёжное хранение артефактов.

Целью курсового проекта является создание распределённой системы генерации фракталов с веб-интерфейсом, поддержкой аутентификации пользователей и хранением истории созданных работ. Архитектура системы построена на принципах микросервисов, что облегчает масштабирование и независимое развитие компонентов [2]. Разработка выполнена в соответствии с принципами объектно-ориентированного проектирования и ориентирована на практическое применение современных инструментов сборки и развёртывания.

Для достижения поставленной цели были решены следующие задачи:

- анализ существующих решений и выявление их сильных и слабых сторон;
- формирование функциональных и нефункциональных требований к системе;
- проектирование микросервисной архитектуры и модели данных;
- выбор технологий реализации, сборки и развёртывания;
- разработка и тестирование ключевых сценариев работы пользователей.

# 1 Анализ аналогов

Для определения требований и позиционирования проекта были рассмотрены наиболее известные решения в области генерации фрактального пламени. Сравнение выполнено по критериям: доступность (desktop/web), качество рендеринга, удобство настройки параметров, наличие истории и механизмов публикации результатов.

## 1.1 Apophysis

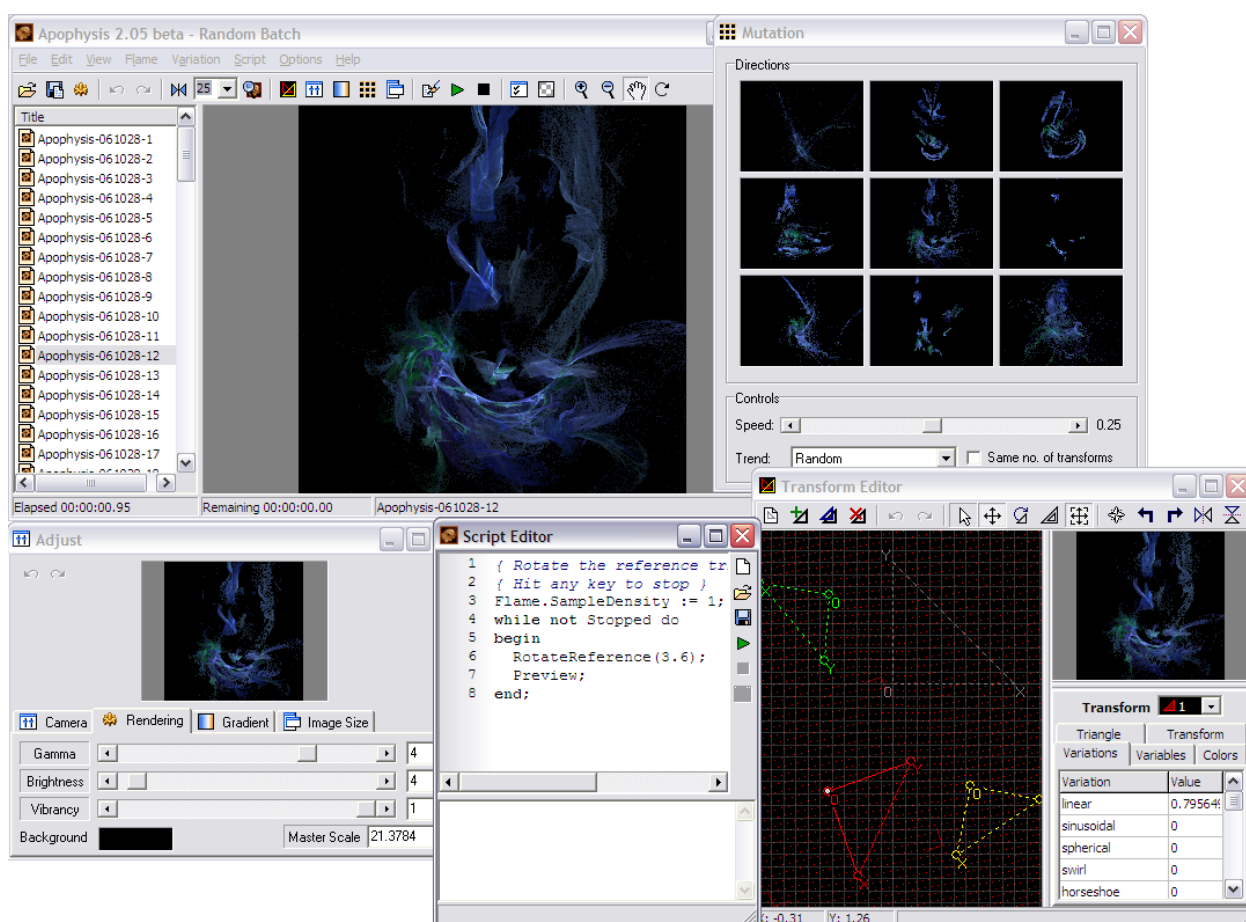


Рисунок 1 – Скриншот пользовательского интерфейса Apophysis с несколькими открытыми окнами

Apophysis — один из наиболее известных редакторов фрактального пламени. Программа распространяется как open-source и предоставляет богатый набор инструментов для ручной настройки трансформаций, палитр и параметров рендера [3]. К достоинствам можно отнести простоту использования и обилие обучающих материалов. Основные ограничения связаны с настольным форматом, отсутствием встроенного хранения истории и невозможностью работы в распределённой среде.

Дополнительно стоит отметить, что основная работа в Apophysis строится вокруг ручного подбора вариаций и параметров, поэтому пользователь часто вынужден сохранять множество промежуточных файлов. Отсутствие центрального хранилища и единой учётной записи делает обмен результатами и совместную работу затруднительными.

## 1.2 JWildfire

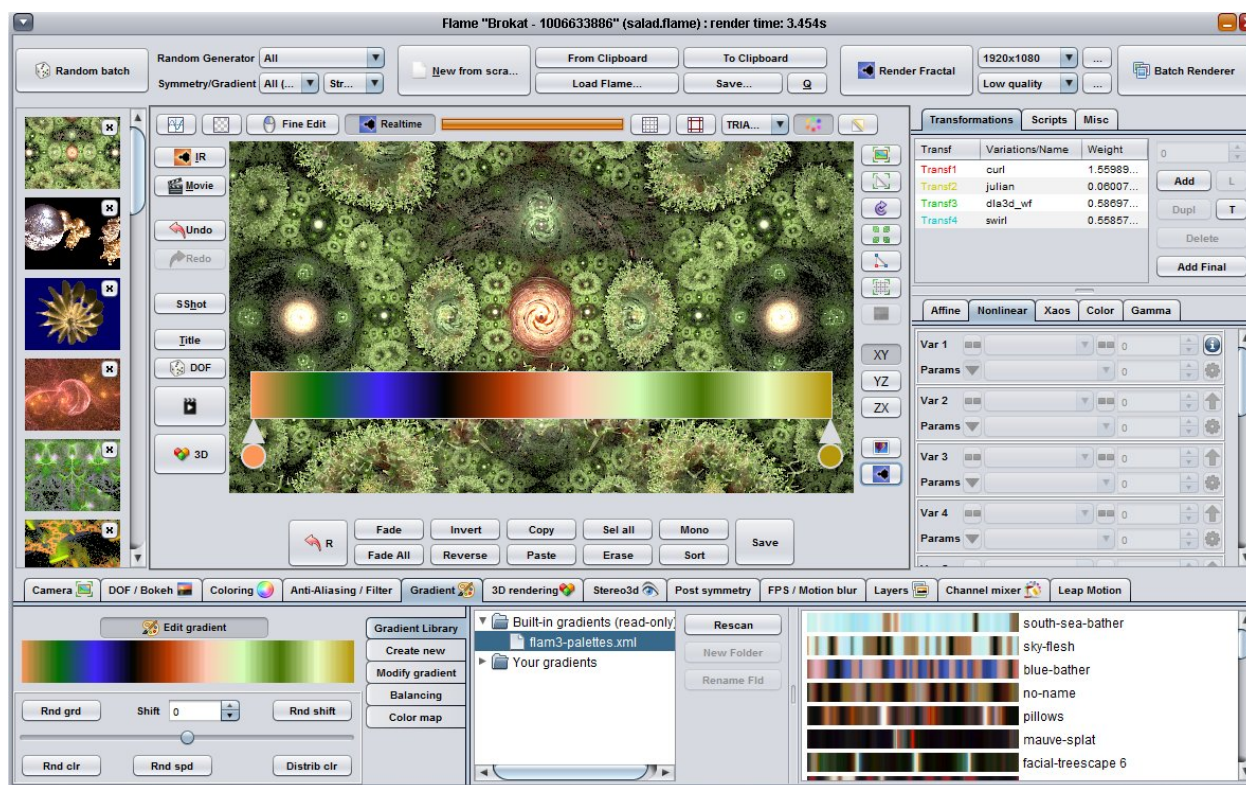


Рисунок 2 – Скриншот пользовательского интерфейса JWildfire

JWildfire — мощный кроссплатформенный редактор фрактального пламени с большим набором вариаций и встроенными генераторами случайных фракталов [4]. Решение предоставляет широкие возможности, но требует значительного времени на освоение и ориентировано на опытных пользователей. Встроенные механизмы совместной работы и веб-публикации отсутствуют, а хранение результатов полностью локально.

Сильной стороной JWildfire является гибкость и глубина настроек, что делает его удобным для экспериментов и профессионального использования. Однако сложный интерфейс и необходимость локальной установки снижают доступность для начинающих пользователей и не решают задачу быстрого доступа к проектам с разных устройств.

## 1.3 Chaotica

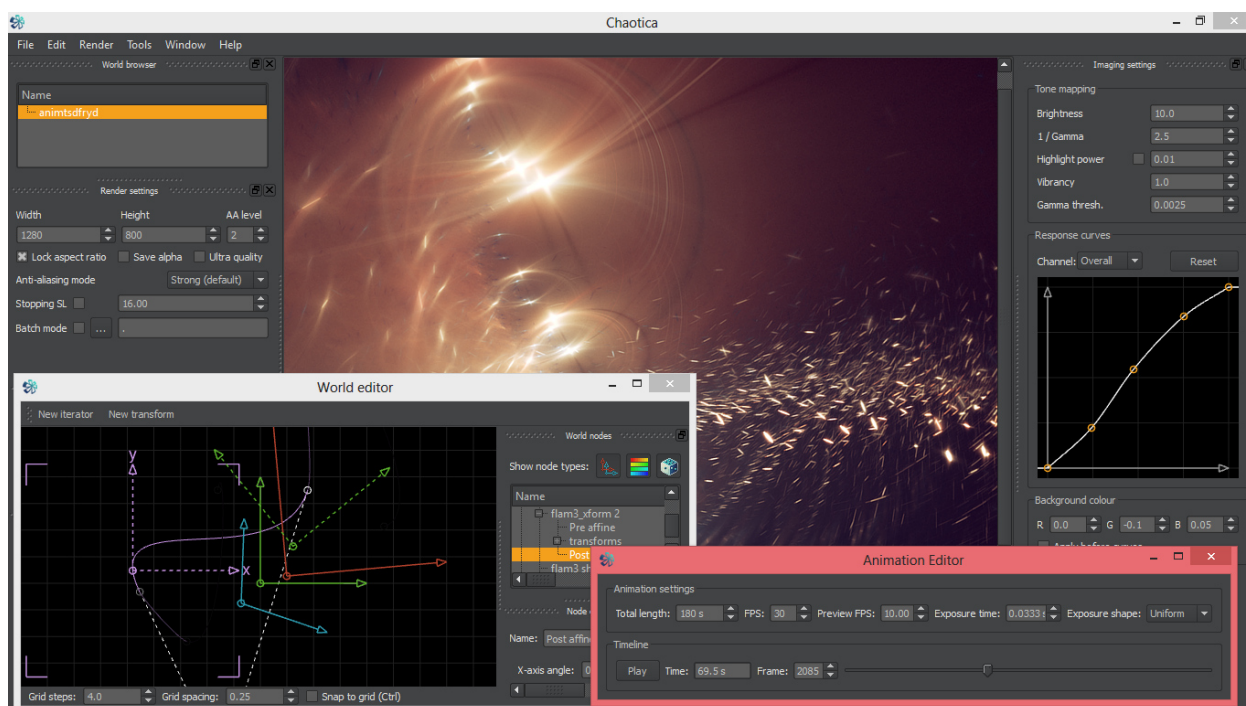


Рисунок 3 – Скриншот анимации фрактала, создаваемой в Chaotica

Chaotica — коммерческое приложение, ориентированное на получение высококачественных изображений и ускоренный рендеринг (включая GPU) [5]. Преимущества: высокая скорость и качество, развитый инструментарий анимации. Недостатки: закрытость, платная модель распространения и ориентация на локальную работу без интегрированного облачного хранения и пользовательских профилей.

В практической эксплуатации продукт удобен для студийных задач и финального качества рендера, однако ориентирован на автономное использование. Это ограничивает возможность модификаций, интеграции с веб-архивом и построения персонализированного профиля пользователя.

## 1.4 Сравнительный анализ

Таблица 1 – Сравнение аналогов по ключевым критериям

| Продукт   | Сильные стороны                                                          | Ограничения                                                             |
|-----------|--------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Arphysics | Простота использования, open-source, богатая база примеров               | Только desktop, нет истории и веб-доступа                               |
| JWildfire | Большое количество вариаций, кросс-платформенность, мощный набор функций | Высокая сложность интерфейса, локальное хранение, отсутствие веб-режима |
| Chaotica  | Высокое качество рендера, поддержка GPU, развитая анимация               | Платная лицензия, закрытость, отсутствие облачной истории               |

Анализ показал, что существующие решения преимущественно ориентированы на локальный рабочий процесс и не обеспечивают удобной работы с историей и публикацией результатов. Это определило требования к проектируемой системе, в частности необходимость веб-интерфейса, учётных записей и долговременного хранения рендеров.

Следовательно, ключевым конкурентным преимуществом будущей системы становится доступность из браузера и единое пространство проектов, где результаты сохраняются и доступны для повторного использования.

## 2 Формирование функциональных требований

На основе анализа аналогов сформированы функциональные требования к системе генерации фракталов:

- регистрация, вход и обновление пользовательской сессии с использованием JWT-токенов;
- создание задачи генерации фрактала с передачей размеров изображения, количества итераций, уровня симметрии, гамма-коррекции, набора функций и аффинных преобразований;
- получение списка доступных функций фрактального пламени;
- отображение прогресса генерации и промежуточного предпросмотра;

- получение списка ранее созданных фракталов с пагинацией, сортировкой и фильтрацией по пользователю;
  - просмотр отдельного фрактала, его параметров и итогового изображения;
  - отображение главной страницы с формой генерации и feed последних сгенерированных фракталов с подписями пользователей;
  - отображение страницы профиля с редактированием пользовательской информации и историей созданных фракталов;
  - сохранение параметров генерации в PostgreSQL и сохранение PNG-изображений в S3-совместимом хранилище MinIO;
  - редактирование базовых данных профиля пользователя.
- Сформированы также нефункциональные требования:
- производительность: приемлемое время рендера при стандартных параметрах качества;
  - масштабируемость: возможность независимо увеличивать количество экземпляров gateway и generator без изменения клиентского приложения;
  - надёжность: сохранность истории генераций в PostgreSQL и изображений в MinIO при перезапуске контейнеров;
  - безопасность: stateless-аутентификация через JWT, передача токена в HTTP-заголовке и проксирование его во внутренние gRPC-вызовы;
  - переносимость: развёртывание всех компонентов через Docker Compose;
  - сопровождаемость: разделение frontend, API gateway, IAM и generator по самостоятельным зонам ответственности;
  - ограничение вычислительной нагрузки: проверка максимальной ширины, высоты и числа итераций на стороне generator.

## 3 Анализ литературы

### 3.1 Принципы объектно-ориентированного проектирования

Объектно-ориентированный подход опирается на понятия инкапсуляции, наследования, полиморфизма и абстракции. Эти принципы позволяют моделировать предметную область в виде взаимосвязанных классов, обеспечивая повторное использование и расширяемость кода [6]. Для разрабатываемой системы это особенно важно, поскольку логика генерации фракталов включает множество разновидностей преобразований, параметров рендера и вариаций, которые удобно представлять в виде иерархий объектов.

В проекте объектные модели описывают параметры рендера, состояние проекта и пользовательский контекст, что позволяет разделить вычислительную и интерфейсную части. Это упрощает развитие

функциональности и делает код более устойчивым к изменениям.

### **3.2 Принципы SOLID и качество кода**

Принципы SOLID ориентированы на снижение связности и повышение устойчивости к изменениям. В частности, разделение ответственности и открытость к расширению позволяют добавлять новые алгоритмы генерации и модули без переработки существующего кода [7]. Эти принципы были использованы при проектировании сервисов, чтобы обеспечивать независимость компонентов и удобство тестирования.

На практике это выразилось в выделении отдельных интерфейсов для рендера, хранения и пользовательских операций, а также в уменьшении числа прямых зависимостей между слоями. Такой подход упростил модульное тестирование и локализацию ошибок.

### **3.3 Шаблоны проектирования**

В литературе по объектно-ориентированному проектированию выделяются шаблоны, упрощающие решение типовых инженерных задач [8]. Для проекта генерации фрактального пламени особенно важны шаблоны, которые позволяют отделить изменяемые алгоритмы от стабильной инфраструктуры приложения.

Шаблон Strategy применим к вариационным функциям фрактального пламени. Каждая функция преобразует точку по собственному правилу, но внешний код генерации может работать с общим интерфейсом функции. Благодаря этому добавление новой вариации не требует переписывать основной цикл рендера. Достаточно реализовать новую модель функции и зарегистрировать её в фабрике или строителе функций.

Шаблон Factory Method полезен при создании доменных объектов на основе входных DTO или protobuf-сообщений. Внешний контракт передаёт имя функции и числовые параметры, а внутри generator должен появиться объект конкретного класса. Вынесение этой логики из сервиса генерации снижает связность и делает ошибочные имена функций отдельным контролируемым сценарием.

Идея Builder применима к созданию задачи генерации. Задача включает размеры изображения, число итераций, симметрию, гамму, набор функций, набор аффинных преобразований и пользовательский контекст. Последовательная сборка такой конфигурации удобнее, чем передача большого количества несвязанных параметров между методами.

### **3.4 Моделирование предметной области**

Для описания предметной области генерации фракталов применяются подходы доменно-ориентированного проектирования. Они позволяют

выделить ключевые сущности и описать их жизненный цикл [9]. В данном проекте основными сущностями являются пользователь, фрактал, вариационная функция, аффинное преобразование, задача генерации и изображение результата.

Сущность фрактала хранит параметры, достаточные для повторного отображения результата: размеры, количество итераций, уровень симметрии, гамму, список функций и список аффинных преобразований. Изображение при этом не смешивается с параметрами: в реляционной базе хранится только имя объекта, а бинарное содержимое помещается в объектное хранилище. Такое разделение поддерживает принцип единственной ответственности на уровне данных.

Пользовательская сущность относится к другому bounded context — IAM. Она не должна зависеть от деталей генерации фракталов. Поэтому generator хранит только внешний идентификатор пользователя, полученный из JWT, а полная информация о профиле остаётся в IAM. Это уменьшает связность микросервисов и упрощает изменение профиля без миграций в сервисе генерации.

### **3.5 Слоистая организация приложения**

Серверные сервисы проекта разделяются на несколько слоёв: транспортный, сервисный, доменный и инфраструктурный. Транспортный слой принимает HTTP или gRPC-запросы и отвечает за преобразование данных из внешнего формата. Сервисный слой координирует сценарии использования: запуск генерации, получение истории, чтение изображения, обновление пользователя. Доменный слой содержит модели фрактала, функций и параметров. Инфраструктурный слой работает с PostgreSQL, MinIO и внешними библиотеками.

Такое разделение важно для тестируемости. Например, алгоритм генерации можно проверять отдельно от gRPC-сервера, а мапперы DTO можно тестировать без запуска базы данных. Кроме того, слой gateway не обязан знать внутреннее устройство generator: он опирается на контракт и выполняет роль адаптера между HTTP-клиентом и внутренними сервисами.

### **3.6 Контракты и переносимость моделей**

В распределённой системе объектная модель существует сразу в нескольких формах: как доменная модель внутри сервиса, как схема базы данных, как protobuf-сообщение для gRPC и как JSON-модель во внешнем HTTP API. Если эти формы изменяются вручную и независимо, быстро появляются расхождения между клиентом и сервером.

Для снижения этого риска используются формальные контракты. gRPC-контракт generator описывает внутренние сообщения и сервисные

методы, а OpenAPI-спецификация описывает внешний HTTP API gateway. Кодогенерация по этим контрактам позволяет получать типы, интерфейсы и модели из единого источника. Это не отменяет ручное проектирование доменной модели, но делает границы между сервисами более проверяемыми.

## 4 Проектирование системы

### 4.1 Общая архитектура

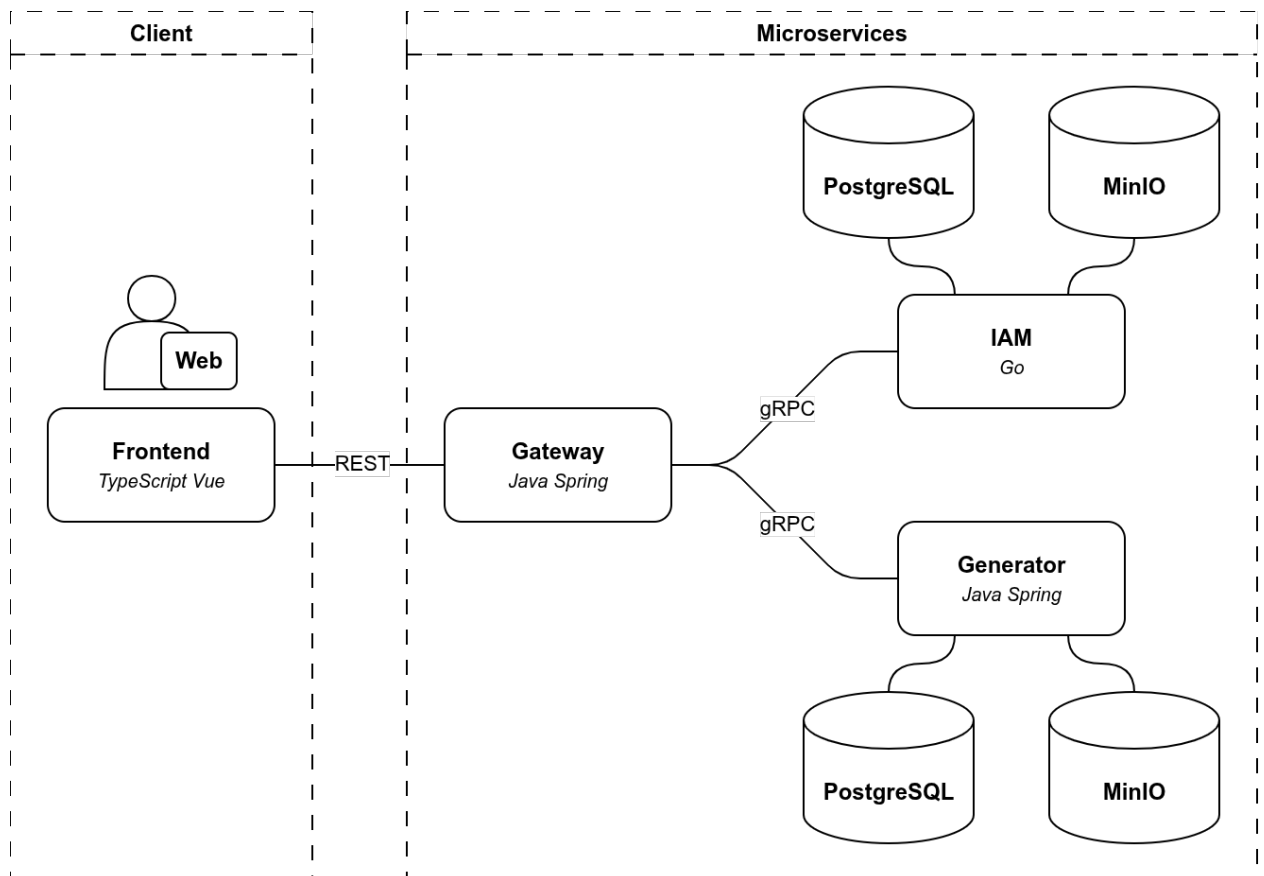


Рисунок 4 – Диаграмма архитектуры

Проект реализуется как распределённая система из четырёх компонентов:

- frontend — клиентское приложение на TypeScript, Vue, Tailwind CSS и Axios;
- API gateway — единая HTTP-точка входа для клиентского приложения, реализованная на Java и Spring;
- IAM — сервис управления пользователями и аутентификацией, реализованный на Go;
- generator — сервис генерации и хранения фракталов, реализованный на Java и Spring.

Frontend взаимодействует только с API gateway по HTTP. Gateway скрывает внутреннюю структуру backend-части и обращается к IAM и generator по gRPC [10]. Такое разделение позволяет независимо развивать пользовательский интерфейс, аутентификацию и вычислительную часть. Внешний контракт остаётся ориентированным на браузерное приложение, а внутренние сервисы могут использовать более строгие protobuf-контракты.

Зоны ответственности компонентов представлены в таблице.

Таблица 2 – Ответственность компонентов системы

| Компонент   | Назначение                                                            | Взаимодействие                                                                    |
|-------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Frontend    | Отображение интерфейса, ввод параметров, просмотр галереи и профиля   | HTTP-запросы к gateway, получение событий прогресса и изображений                 |
| API gateway | Единая точка входа и адаптер между HTTP API и внутренними сервисами   | HTTP с frontend, gRPC-вызовы IAM и generator, передача JWT во внутренние metadata |
| IAM         | Регистрация, вход, обновление токенов и хранение профиля пользователя | gRPC API для gateway, хранение пользовательской записи в PostgreSQL               |
| Generator   | Генерация фрактального пламени, история работ, хранение изображений   | gRPC API для gateway, PostgreSQL для параметров, MinIO для PNG-файлов             |

## 4.2 Frontend

Frontend отвечает за пользовательский сценарий работы с системой. Он реализуется как одностраничное приложение, в котором выделены две основные страницы: главная страница генерации и страница профиля. Главная страница объединяет форму параметров фрактала, блок текущего состояния задачи и feed последних сгенерированных работ. В feed отображается миниатюра результата, краткие параметры и подпись пользователя, создавшего фрактал. Галерея примеров результатов должна быть приведена в приложении А.

Страница профиля решает другую группу задач: редактирование пользовательской информации и просмотр личной истории генераций. В верхней части страницы размещается форма изменения имени и изображения профиля, а ниже — список фракталов текущего пользователя. Такое разделение интерфейса поддерживает два режима работы: быстрый

творческий цикл на главной странице и управление личным архивом на странице профиля.

Клиентский код не обращается напрямую к IAM или generator. Все запросы идут через API gateway, что упрощает хранение базового URL, обработку ошибок и передачу JWT. HTTP-контракт описывается OpenAPI-спецификацией [11]; на стороне frontend она используется как источник типизированных моделей и клиентских функций. Благодаря этому изменение структуры запроса или ответа быстрее обнаруживается на этапе сборки.

### 4.3 API gateway

API gateway является границей backend-части. Он предоставляет frontend HTTP-интерфейс в формате JSON и скрывает внутренние gRPC-сервисы. Для работы с фракталами предусмотрены маршруты получения списка фракталов, получения фрактала по идентификатору, запуска генерации, подписки на события задачи, получения списка функций и получения PNG-изображения. Для пользовательских сценариев предусмотрены маршруты регистрации, входа, обновления токена, получения и изменения профиля.

В gateway внешний API описан в файле `openapi.yaml`. По этой спецификации OpenAPI Generator формирует Java-модели и API-интерфейсы, которые затем реализуются контроллерами. Это снижает вероятность расхождения между документацией, серверной реализацией и клиентским кодом.

Внутри gateway слой HTTP-контроллеров отделён от слоя вызовов downstream-сервисов. Контроллер `FractalsController` принимает запросы на маршрутах `/api/v1/fractals`, `/api/v1/fractals/gen`, `/api/v1/fractals/functions` и `/api/v1/fractals/images/{name}`. Сервисный слой использует gRPC-stub'ы generator: блокирующий stub применяется для коротких операций, а асинхронный stub — для подписки на состояние задачи.

Gateway также отвечает за передачу пользовательского контекста. JWT приходит от frontend в HTTP-заголовке `Authorization`, после чего gateway передаёт токен во внутренние gRPC metadata. В результате generator может связать созданный фрактал с пользователем, не имея прямого HTTP-взаимодействия с браузером.

### 4.4 IAM

IAM выделен в отдельный микросервис, чтобы логика пользователей и токенов не смешивалась с генерацией изображений. Сервис предоставляет внутренний gRPC API для регистрации, входа, обновления токена,

получения пользователя и изменения профиля. Внешние HTTP-маршруты остаются на gateway, поэтому frontend не зависит от конкретной реализации IAM.

Аутентификация построена на JWT [12]. После успешного входа IAM выдаёт access token и refresh token. Access token используется frontend при обращении к gateway, а refresh token позволяет обновлять пользовательскую сессию без повторного ввода пароля. Токен содержит идентификатор пользователя, который downstream-сервисы используют как связку между действиями пользователя и данными системы.

Для IAM предусмотрена отдельная модель пользователей. В рамках текущих требований она состоит из одной таблицы users, где хранятся идентификатор пользователя, имя пользователя, хеш пароля и ссылка на изображение профиля или имя объекта в хранилище. Идентификатор пользователя используется как subject JWT и передаётся downstream-сервисам для связывания пользовательских действий с историей генераций.

## 4.5 Generator

Generator отвечает за вычислительно тяжёлую часть системы: создание фрактального пламени, хранение параметров генерации, хранение результата и выдачу истории работ. Внешние клиенты не обращаются к нему напрямую; доступ выполняется через API gateway по внутреннему gRPC-контракту.

Основной контракт generator описан в файле generator.proto. Сервис Fractals предоставляет следующие операции:

- GetAll — получение списка фракталов с пагинацией, сортировкой и фильтрацией по user\_id;
- GetById — получение одного фрактала по идентификатору;
- Generation — создание задачи генерации;
- SubscribeToTask — серверный поток состояния задачи;
- GetFunctions — получение доступных имён вариационных функций;
- GetImage — чтение изображения из S3-совместимого хранилища.

Запрос генерации содержит ширину и высоту изображения, число итераций, уровень симметрии, параметр гамма-коррекции, список вариационных функций и список аффинных преобразований. Перед постановкой задачи в очередь generator проверяет ограничения параметров: максимальную ширину, высоту, число итераций, положительный уровень симметрии, наличие хотя бы одной функции и одного набора аффинных параметров.

Сервис также содержит каталог поддерживаемых функций фрактального пламени. По коду generator доступны функции linear, sinusoidal, spherical, swirl, horseshoe, polar, disc, heart и

eyefish. Каждая функция представлена отдельной моделью, что упрощает добавление новых вариаций без изменения внешнего API.

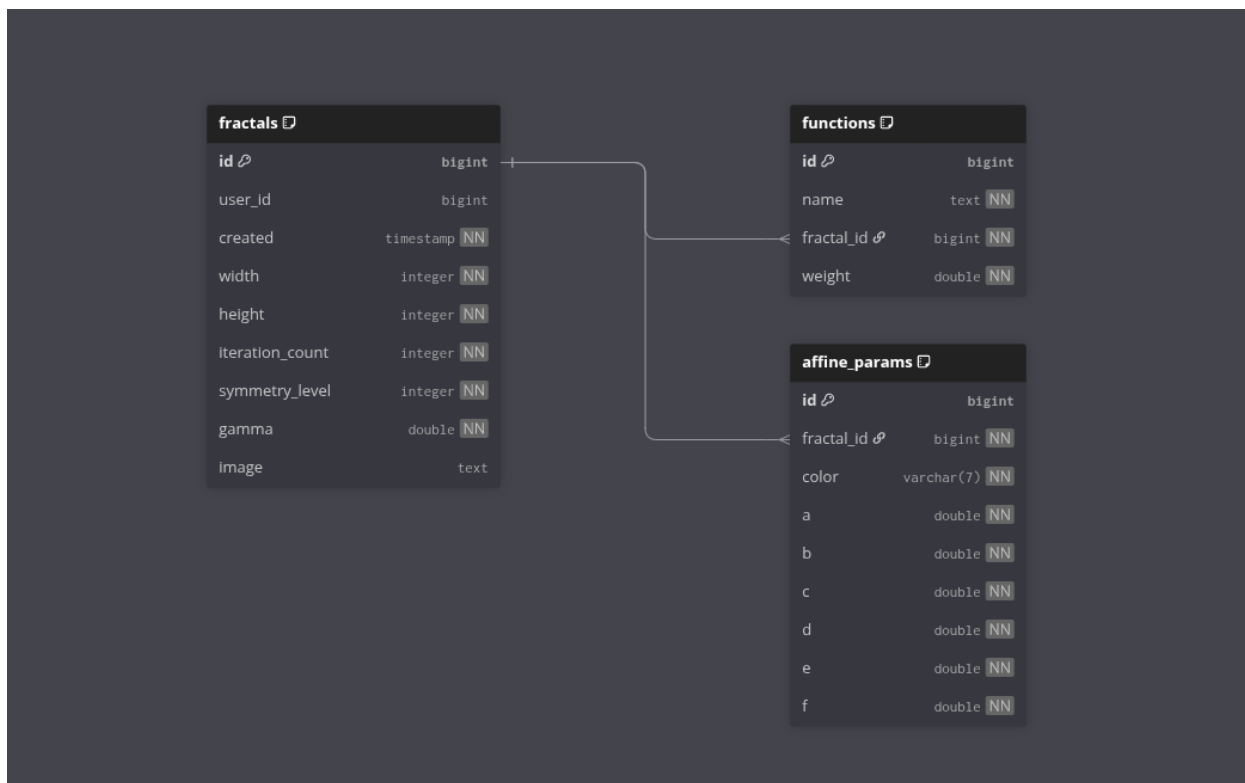


Рисунок 5 – ER-диаграмма сервиса генерации

Модель данных generator строится вокруг истории созданных фракталов. По миграциям Liquibase используются три таблицы: `fractals`, `functions` и `affine_params`. Таблица `fractals` хранит общие параметры результата: идентификатор, идентификатор пользователя, дату создания, размеры изображения, число итераций, уровень симметрии, гамму и имя файла изображения. Таблица `functions` хранит набор вариационных функций и веса для конкретного фрактала. Таблица `affine_params` хранит цвет и коэффициенты аффинного преобразования  $a$ ,  $b$ ,  $c$ ,  $d$ ,  $e$ ,  $f$ .

Связи имеют тип «один ко многим»: один фрактал содержит несколько функций и несколько наборов аффинных параметров. Для дочерних таблиц настроено каскадное удаление по внешнему ключу `fractal_id`, поэтому удаление фрактала не оставляет несвязанных параметров. Для ускорения выборки созданы индексы по `fractals.user_id`, `functions.fractal_id` и `affine_params.fractal_id`.

Метаданные фракталов хранятся в PostgreSQL, а бинарные PNG-файлы вынесены в MinIO — локальное S3-совместимое объектное хранилище [13]. Такое разделение уменьшает нагрузку на реляционную базу данных и упрощает выдачу файлов. Generator использует AWS S3 SDK, создаёт `bucket images` при запуске, сохраняет PNG с типом `image/png` и возвращает имя

файла, которое затем хранится в таблице `fractals`.

При запросе изображения `gateway` вызывает `gRPC`-метод `GetImage`; `generator` читает объект из `MinIO` и возвращает байтовое содержимое. Для `frontend` это остаётся обычным `HTTP`-ответом с изображением, а внутренняя реализация хранения скрыта за `gateway`.

## 4.6 Процесс генерации фрактального пламени

Генерация фрактального пламени строится как итерационный процесс построения изображения по набору аффинных преобразований и вариационных функций. Пользователь передаёт размеры изображения, количество итераций, уровень симметрии, гамму, веса функций и коэффициенты аффинных преобразований. Эти параметры сохраняются как часть доменной модели фрактала, после чего на их основе создаётся объект задачи `GenerationTask`. Фрагменты кода генератора и сервиса генерации должны быть приведены в приложении Б.

Внутри рендера выбирается начальная случайная точка и случайный цвет. На каждой итерации выбирается одно аффинное преобразование и одна вариационная функция. Аффинное преобразование переносит точку в новое положение, а вариационная функция применяет нелинейное преобразование. Затем точка дополнительно обрабатывается с учётом уровня симметрии: для каждого симметричного поворота рассчитываются координаты пикселя на плоскости изображения.

Первые итерации используются как прогрев и не записываются в изображение, чтобы начальное случайное состояние меньше влияло на результат. После прогрева координаты точки преобразуются в координаты пикселя, цвет смешивается с цветом текущего аффинного преобразования, а счётчик посещений пикселя увеличивается. Чем чаще точка попадает в область изображения, тем сильнее эта область проявляется после нормализации.

Для ускорения вычислений `generator` делит работу между несколькими потоками. Класс `Generator` создаёт пул потоков, а каждый экземпляр `Render` выполняет свою часть итераций над общей моделью изображения. Доступ к пиксельной матрице синхронизируется, поскольку несколько потоков могут одновременно обновлять один и тот же результат. После завершения потоков выполняется гамма-коррекция: счётчики посещений переводятся в логарифмическую нормализацию, после чего цвет пикселей усиливается с учётом параметра `gamma`.

Результат генерации сохраняется в `MinIO` как `PNG`-файл, а в `PostgreSQL` остаётся ссылка на объект и параметры, достаточные для отображения истории. Такой подход делает результат воспроизводимым на уровне параметров и одновременно не перегружает реляционную базу бинарными данными.

## 4.7 Передача прогресса генерации

Генерация фрактала может занимать заметное время, поэтому пользователь не должен ждать завершения HTTP-запроса без обратной связи. Для этого генерация реализована как асинхронная задача. Метод `Generation` в `generator` сохраняет параметры, создаёт `GenerationTask`, добавляет её в очередь и сразу возвращает начальное состояние задачи с идентификатором фрактала и текущим прогрессом.

Внутри `GeneratationService` используется очередь `BlockingQueue` и набор ожидающих задач `ConcurrentHashMap`. При запуске сервиса создаётся фиксированный пул `worker`-потоков. Каждый `worker` берёт задачу из очереди, вызывает `generator.process(task)`, после завершения сохраняет PNG-изображение в `MinIO`, обновляет запись фрактала в `PostgreSQL` и удаляет задачу из списка ожидающих.

Прогресс считается в `GenerationTask`. Внутри задачи хранится счётчик выполненных итераций `AtomicLong`; поток рендера периодически увеличивает его пакетами итераций. Метод `toTaskState()` формирует `protobuf`-сообщение `TaskState`: идентификатор фрактала, число прогресса от 0 до 1 и `base64`-предпросмотр текущего изображения.

Рассылка обновлений выполняется через `scheduler Spring`. Метод `tasksUpdate()` помечен аннотацией `@Scheduled(fixedDelayString = "${app.tasks-update-delay})`; в конфигурации значение задержки задано как `1s`. `Scheduler` проходит по ожидающим задачам и вызывает `sendUpdate(task)`. Если для задачи есть подписчик, сервис отправляет очередное `TaskState` в `gRPC stream`.

`Gateway` подписывается на `gRPC`-метод `SubscribeToTask` асинхронным `stub`-ом. Каждое сообщение `TaskState`, пришедшее от `generator`, преобразуется в модель внешнего API и отправляется браузеру через `SseEmitter`. Для `frontend` это выглядит как поток `text/event-stream`, то есть `Server-Sent Events` [14]. Когда прогресс достигает 1, `gateway` завершает `SSE`-поток. Если `gRPC stream` завершается ошибкой, `SseEmitter` также завершается с ошибкой, чтобы клиент мог показать пользователю корректное состояние.

## 4.8 Развёртывание

Развёртывание выполняется через `Docker Compose` [15]. Каждый сервис запускается в отдельном контейнере, а инфраструктурные зависимости поднимаются рядом с приложением. Для `generator` используются `PostgreSQL` и `MinIO`; контейнер `generator` получает адрес `PostgreSQL` через переменную `POSTGRES_URL`, адрес `MinIO` через `MINIO_URL`, а секрет `JWT` через `JWT_SIGNING_KEY`. `Gateway` получает адрес `generator` через `GENERATOR_URL`; аналогично для полной конфигурации задаётся адрес

IAM.

Compose-сценарий содержит healthcheck'и для базы данных, MinIO, gateway и generator. Это снижает вероятность запуска приложения до готовности зависимостей и делает локальное окружение воспроизводимым. На сервере запускается тот же Compose-проект, что используется при локальной проверке; отличаются только значения переменных окружения, доменное имя и настройки обратного прокси.

Финальная версия системы разворачивается в Docker-контейнерах на VPS. Для размещения серверной части использовался хостинг-провайдер Jino.ru, выбранный из-за SSH-доступа, возможности управления сервером и интеграции доменов с SSL-сертификатами [16]. В составе развёртывания применяется обратный прокси для маршрутизации запросов и разграничения доступа к сервисам.

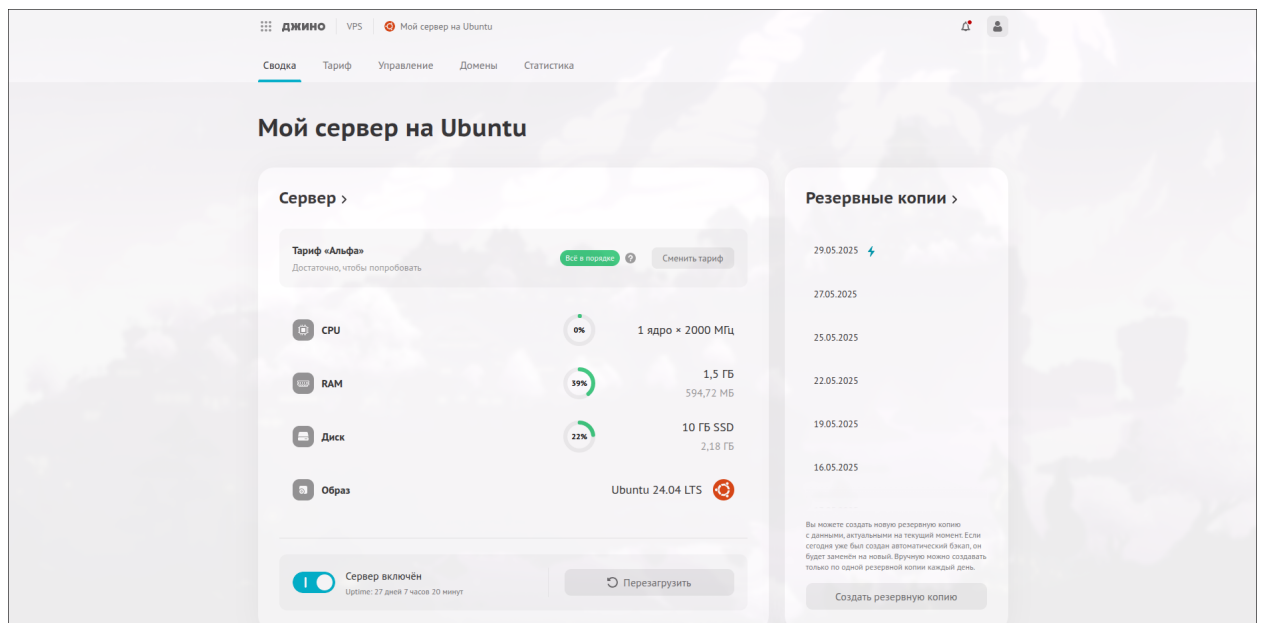


Рисунок 6 – Страница сервера на jino.ru

## 4.9 Технологии используемые для решения поставленных задач

### 4.9.1 Язык программирования Java



Рисунок 7 – Java

Язык программирования Java является объектно-ориентированным языком, разработанным компанией Sun Microsystems и в настоящее время поддерживаемым корпорацией Oracle. Java обеспечивает высокий уровень абстракции, платформенную независимость и безопасность, что делает его одним из самых популярных языков в мире.

Одним из основных преимуществ Java является его независимость от платформы, достигаемая благодаря использованию виртуальной машины Java (JVM), которая исполняет байт-код, одинаковый для всех операционных систем. Это свойство позволяет создавать переносимые приложения без необходимости модификации исходного кода.

Java поддерживает основные принципы объектно-ориентированного программирования, такие как наследование, инкапсуляция, полиморфизм и абстракция. Эти особенности обеспечивают высокую модульность и повторное использование кода. [17]

В рамках проекта Java используется в двух сервисах: API gateway и generator. Для этих компонентов язык выбран благодаря следующим преимуществам:

- наличие развитой экосистемы для HTTP API, gRPC, работы с базами данных и контейнеризированного запуска;
- развитая экосистема, включающая библиотеки и фреймворки, ускоряющие процесс разработки;
- встроенная система управления памятью с автоматическим сборщиком мусора, упрощающая разработку и снижающая вероятность ошибок, связанных с управлением ресурсами;
- высокая читаемость и строгая типизация, что способствует написанию надёжного и легко сопровождаемого кода.

В gateway Java применяется для обработки HTTP-запросов, настройки фильтра JWT и вызова gRPC-сервисов. В generator язык используется для реализации алгоритма фрактального пламени, очереди задач, сохранения метаданных и работы с S3-совместимым хранилищем.

#### 4.9.2 Фреймворк Spring



Рисунок 8 – Spring Framework

Spring — это мощный и гибкий фреймворк с открытым исходным кодом для разработки корпоративных и веб-приложений на языке Java. Благодаря своей модульной структуре, высокой производительности и удобству масштабирования, Spring широко применяется при создании распределённых и высоконагруженных систем, включая веб-сервисы и многопользовательские онлайн-платформы.

Одной из ключевых особенностей Spring является внедрение зависимостей (Dependency Injection), которое способствует слабой связанности компонентов приложения и облегчает их тестирование, повторное использование и расширение. Фреймворк построен по принципу инверсии управления (Inversion of Control), где управление жизненным циклом объектов осуществляется специальным контейнером.

В проекте Spring используется в gateway и generator. Gateway принимает HTTP-запросы от frontend, а generator предоставляет gRPC API для генерации фракталов и доступа к истории.

В процессе разработки были использованы следующие модули Spring:

- Spring Boot — модуль, предназначенный для упрощения конфигурации и быстрой инициализации проекта. Он предоставляет встроенный сервер и автоматическую настройку компонентов, что позволило сосредоточиться на реализации логики приложения;
- Spring Web — модуль, реализующий обработку HTTP-запросов, маршрутизацию и построение REST API gateway;
- Spring gRPC — модуль для реализации gRPC-сервера в generator и gRPC-клиента в gateway;
- Spring Data JPA — модуль для работы с PostgreSQL через объектно-реляционное отображение;

– Spring Security — модуль, обеспечивающий обработку JWT в HTTP-запросах gateway и gRPC metadata generator.

Использование Spring позволило добиться высокой скорости обработки запросов, надёжности и масштабируемости серверной части. Благодаря модульной архитектуре фреймворка, разработка велась по принципу разделения ответственности, что улучшило структуру проекта и упростило поддержку и развитие функционала. [18]

Дополнительно применяются встроенные механизмы валидации данных, централизованная обработка ошибок и healthcheck'и, что обеспечивает единый формат ответов и предсказуемость поведения сервисов.

#### 4.9.3 Язык Go

Язык Go используется для реализации IAM-сервиса. Для сервиса аутентификации важны быстрый запуск, простая модель конкурентного выполнения, компактная поставка и удобная работа с сетевыми сервисами. Go хорошо подходит для небольших backend-компонентов, которые предоставляют API, работают с базой данных и не требуют тяжёлой runtime-среды [19].

В рамках проекта IAM отвечает за регистрацию пользователей, проверку пароля, выпуск JWT и обновление сессии. Выделение этого сервиса на Go позволяет развивать пользовательскую часть независимо от Java-сервисов генерации.

#### 4.9.4 Frontend: TypeScript, Vue, Tailwind CSS и Axios

Frontend реализуется как одностраничное веб-приложение. TypeScript добавляет статическую типизацию поверх JavaScript и помогает согласовать модели запросов и ответов с OpenAPI-спецификацией [20]. На основе OpenAPI-контракта формируются типы и клиентские функции для обращения к gateway, поэтому изменения HTTP API быстрее обнаруживаются на этапе сборки frontend. Vue выбран как фреймворк для построения реактивного интерфейса: он упрощает разбиение экрана на компоненты, управление состоянием формы генерации и обновление истории факталоов [21].

Tailwind CSS применяется для быстрой сборки адаптивного интерфейса из утилитарных классов [22]. Такой подход удобен для страниц с большим количеством параметров генерации: размеры, отступы, состояния кнопок и сетки можно задавать непосредственно рядом с разметкой компонента.

Axios используется как HTTP-клиент для обращений frontend к API gateway [23]. Через него выполняются запросы регистрации и входа,

получение списка фракталов, запуск генерации и загрузка изображений. Для прогресса рендера применяется поток событий `text/event-stream`.

#### 4.9.5 Система сборки Gradle

Gradle используется для сборки Java-сервисов `gateway` и `generator`. Он обеспечивает управление зависимостями, генерацию Java-кода из `protobuf`-контрактов, запуск тестов и сборку исполняемых приложений. В `gateway` дополнительно подключён `OpenAPI Generator`: из файла `openapi.yaml` формируются Java-модели и API-интерфейсы, которые затем реализуются контроллерами сервиса [24].

#### 4.9.6 База данных PostgreSQL



Рисунок 9 – PostgreSQL

Для хранения структурированных данных используется система управления базами данных PostgreSQL [25]. Она выбрана из-за поддержки транзакций, ограничений целостности, индексов и зрелой интеграции с backend-стеком. Конкретная схема данных `generator` и IAM описана выше в подразделах о моделях данных, поэтому в данном месте фиксируется только обоснование выбора СУБД.

#### 4.9.7 MinIO и S3-совместимое хранилище

Для хранения изображений используется MinIO — S3-совместимое объектное хранилище [13]. Оно позволяет отделить бинарные PNG-файлы от реляционных данных и оставить PostgreSQL для метаданных, связей и поиска. Благодаря совместимости с S3 при необходимости локальное хранилище можно заменить внешним объектным хранилищем без изменения пользовательских сценариев.

#### 4.9.8 gRPC

gRPC применяется для внутреннего взаимодействия gateway с generator и IAM [10]. По сравнению с передачей JSON между backend-сервисами protobuf-контракты дают строгую схему сообщений, компактное представление данных и удобную генерацию клиентского и серверного кода. Подробный сценарий потоковой передачи прогресса рассмотрен в подразделе о внутренних gRPC-контрактах.

#### 4.9.9 JWT

JSON Web Token используется как формат access token [12]. Токен содержит идентификатор пользователя и имя, подписывается секретным ключом и проверяется без обращения к базе данных при каждом запросе. Благодаря этому gateway остаётся stateless, а механизм передачи пользовательского контекста между сервисами не требует общей HTTP-сессии.

#### 4.9.10 Контейнеризация Docker и Docker Compose

Для развёртывания использовалась контейнеризация Docker. Каждый микросервис упакован в отдельный образ, а общий сценарий запуска описан в Docker Compose [15]. Compose выбран как достаточный для курсового проекта способ поднять приложение, базу данных и MinIO одной командой в воспроизводимом локальном или серверном окружении.

## 4.9.11 Среда разработки Visual Studio Code

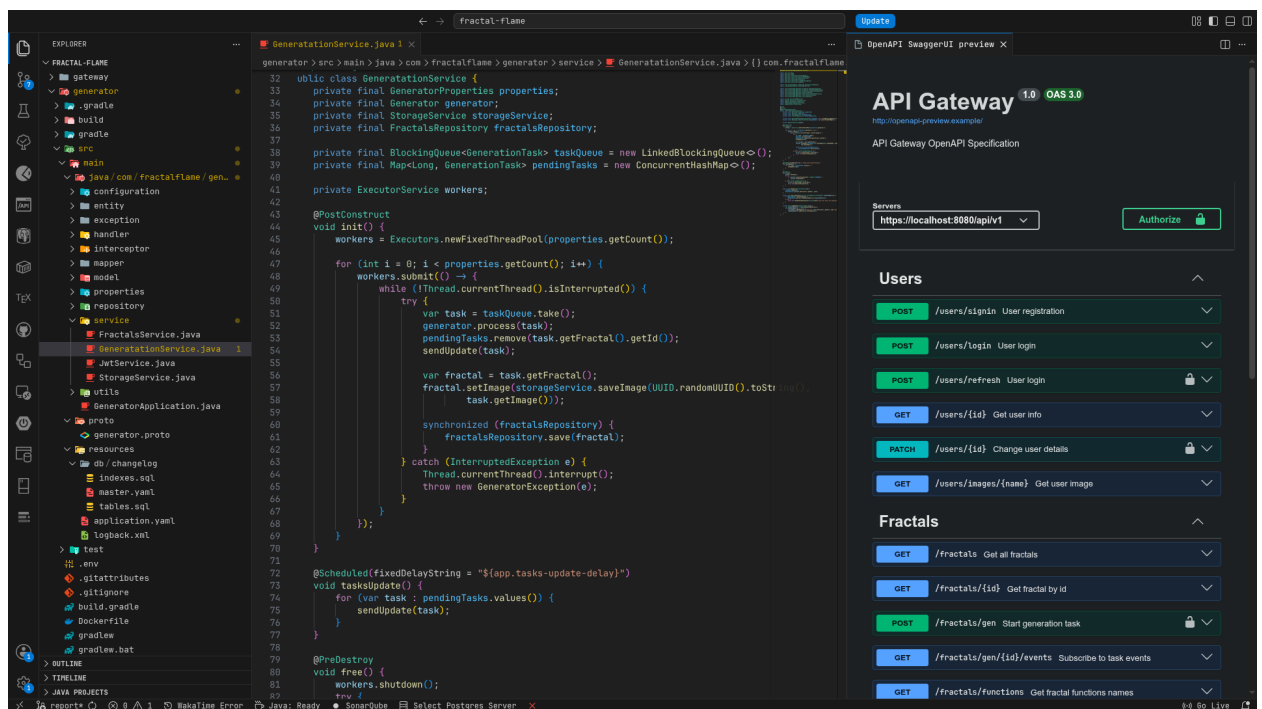


Рисунок 10 – Экран редактирования кода Visual Studio Code

Visual Studio Code (VS Code) — это лёгкая, но мощная интегрированная среда разработки, разработанная компанией Microsoft. С момента своего выхода в 2015 году она завоевала огромную популярность благодаря простоте, скорости и гибкости. VS Code является кроссплатформенным инструментом, доступным на Windows, macOS и Linux, что делает его универсальным решением для разработчиков по всему миру. [26]

VS Code не перегружена лишними функциями, но в то же время предоставляет всё необходимое для разработки: интеллектуальную подсветку синтаксиса, автодополнение, отладку, поддержку Git, систему контроля версий и встроенный терминал. Это позволяет разрабатывать и тестировать код быстро и эффективно, без лишних задержек.

Основные причины выбора VS Code включают:

1. Лёгкость и быстроедействие. VS Code быстро запускается и работает с минимальными задержками, что делает её идеальной для разработки фронтенд-части и быстрого прототипирования.
2. Расширяемость. Благодаря множеству доступных расширений, VS Code легко адаптируется под любые задачи.
3. Интеграция с Git и другими системами контроля версий. VS Code имеет встроенную поддержку Git, что упрощает работу с репозиториями и позволяет эффективно управлять версиями проекта.

Разработка всех компонентов проекта выполнялась исключительно в

Visual Studio Code, что обеспечило единообразие среды и повторяемость процесса. Дополнительно использовались расширения для Java, Docker и работы с  $\text{\LaTeX}$ , что позволило вести и разработку, и оформление документации в одном инструменте. Даже этот документ написан в VS Code с использованием  $\text{\LaTeX}$ .

## 5 Тестирование программного обеспечения

### 5.1 Стратегия тестирования

Тестирование системы проводилось на нескольких уровнях: модульном, интеграционном и системном. На модульном уровне проверялась корректность отдельных компонентов генерации и обработки параметров. Интеграционные тесты подтверждали корректность взаимодействия микросервисов, а системные тесты моделировали реальную работу пользователя. Подход соответствует классическим рекомендациям по организации тестирования программного обеспечения [27].

Для проверки критических сценариев использовались контрольные наборы параметров рендера и заранее подготовленные эталонные результаты. Это позволило выявлять регрессии при изменениях алгоритмов и конфигурации.

### 5.2 Сценарии тестирования

Ниже приведены основные сценарии, подтверждающие работоспособность системы:

1. Регистрация и вход пользователя: создание записи в IAM, получение access token и refresh token.
2. Проверка JWT: передача access token из frontend в gateway и дальнейшая передача токена во внутренний gRPC-вызов.
3. Создание задачи генерации: отправка параметров фрактала через HTTP API gateway и получение идентификатора задачи.
4. Подписка на прогресс: получение событий text/event-stream, сформированных gateway на основе gRPC stream generator.
5. Сохранение результата: запись параметров фрактала в PostgreSQL и PNG-изображения в MinIO.
6. Просмотр истории: получение списка фракталов с пагинацией, сортировкой и фильтрацией по пользователю.
7. Получение изображения: запрос изображения через gateway и чтение объекта из MinIO сервисом generator.
8. Проверка ограничений параметров: отказ generator при превышении максимальных размеров или количества итераций.

Результаты тестирования подтвердили корректность работы ключевых функций и соответствие заявленным требованиям.

Отдельно фиксировались временные показатели рендера и устойчивость к отказам отдельных сервисов, что подтвердило работоспособность системы в условиях повышенной нагрузки.

## ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта получена проектная и программная основа веб-системы для генерации изображений фрактального пламени. Система закрывает основной недостаток рассмотренных аналогов: вместо локального desktop-инструмента пользователь получает веб-интерфейс с запуском генерации, просмотром прогресса, сохранением результата, общей галереей последних работ и личной историей. Примеры итоговой галереи фракталов вынесены в приложение А.

Главным инженерным результатом стала микросервисная архитектура из четырёх компонентов: frontend, API gateway, IAM и generator. Frontend работает с gateway по HTTP, а gateway взаимодействует с IAM и generator по gRPC. Такое разделение позволило отделить пользовательский интерфейс от внутренней логики аутентификации и вычислений. Внешний API зафиксирован OpenAPI-спецификацией, а внутренний контракт generator описан protobuf-схемой, что уменьшает риск расхождения между клиентом, gateway и backend-сервисами.

Отдельным результатом стала реализация сервиса generator. Он принимает параметры фрактального пламени, проверяет ограничения, сохраняет модель фрактала, ставит задачу в очередь, выполняет многопоточный рендер, применяет гамма-коррекцию и сохраняет PNG-изображение в MinIO. Метаданные результата сохраняются в PostgreSQL, а связь с пользователем выполняется через идентификатор из JWT. Код генератора и сервиса генерации должен быть приведён в приложении Б.

Для пользовательского опыта был решён сценарий длительной операции генерации. Generator хранит состояние задачи, scheduler периодически формирует обновления прогресса, gRPC stream передаёт их в gateway, а gateway преобразует поток во внешний SSE-канал для браузера. Благодаря этому пользователь видит не только факт запуска задачи, но и промежуточный предпросмотр изображения вместе с числовым прогрессом.

Также получены результаты по хранению и развёртыванию. PostgreSQL используется для структурированных данных IAM и generator, MinIO — для изображений, Docker Compose — для воспроизводимого запуска сервисов и инфраструктуры. Подготовленная схема развёртывания позволяет запускать один и тот же набор компонентов локально и на VPS, меняя только переменные окружения и настройки прокси.

Таким образом, цель курсового проекта достигнута: спроектирована и реализована основа распределённого приложения, поддерживающего регистрацию и вход пользователя, генерацию фракталов, передачу прогресса, сохранение истории, выдачу изображений и просмотр результатов через веб-интерфейс. Полученное решение остаётся расширяемым: в дальнейшем можно добавить новые вариационные функции, роли пользователей, очередь фоновых задач, более гибкий

редактор параметров и дополнительные способы публикации готовых изображений.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Draves, S. The Fractal Flame Algorithm / S. Draves, E. Reckase // The Flame Algorithm. — 2003. — 41 P.
- [2] Newman, Sam. Building Microservices / Sam Newman. — 2 edition. — O'Reilly Media, 2021.
- [3] Apophysis: Fractal Flame Editor [Электронный ресурс]. — Режим доступа : <https://sourceforge.net/projects/apophysis/>.
- [4] JWildfire: Fractal Flame Generator (Repository) [Электронный ресурс]. — Режим доступа : <https://github.com/thargor6/JWildfire>.
- [5] Chaotica Fractal Flame Software [Электронный ресурс]. — Режим доступа : <https://www.chaoticafractals.com/>.
- [6] Booch, G. Object-Oriented Analysis and Design with Applications / G. Booch. — 3 edition. — Addison-Wesley Professional, 2007.
- [7] Martin, Robert C. Agile Software Development: Principles, Patterns, and Practices / Robert C. Martin. — Prentice Hall, 2002.
- [8] Design Patterns: Elements of Reusable Object-Oriented Software / E. Gamma, R. Helm, R. Johnson, J. Vlissides. — Addison-Wesley, 1994.
- [9] Evans, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software / Eric Evans. — Addison-Wesley, 2003.
- [10] gRPC Documentation [Электронный ресурс]. — Режим доступа : <https://grpc.io/docs/>.
- [11] OpenAPI Specification [Электронный ресурс]. — Режим доступа : <https://spec.openapis.org/oas/latest.html>.
- [12] Jones, M. JSON Web Token (JWT): Tech. Rep. RFC 7519 / M. Jones, J. Bradley, N. Sakimura: IETF, 2015. — Режим доступа : <https://www.rfc-editor.org/rfc/rfc7519>.
- [13] MinIO Object Store Documentation [Электронный ресурс]. — Режим доступа : <https://min.io/docs/minio/linux/index.html>.
- [14] Server-sent events — MDN Web Docs [Электронный ресурс]. — Режим доступа : [https://developer.mozilla.org/en-US/docs/Web/API/Server-sent\\_events](https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events).
- [15] Docker Compose Documentation [Электронный ресурс]. — Режим доступа : <https://docs.docker.com/compose/>.
- [16] Jino.ru: виртуальный хостинг и VPS [Электронный ресурс]. — Режим доступа : <https://jino.ru/>.
- [17] Schildt, H. Java: The Complete Reference, Ninth Edition / H. Schildt. — McGraw-Hill Education, 2014. — Pp. 17 – 23.
- [18] Walls, Craig. Spring in Action / Craig Walls. — 6 edition. — Manning Publications, 2022.
- [19] The Go Programming Language Documentation [Электронный ресурс]. — Режим доступа : <https://go.dev/doc/>.
- [20] TypeScript Documentation [Электронный ресурс]. — Режим доступа :

- <https://www.typescriptlang.org/docs/>.
- [21] Vue.js Documentation [Электронный ресурс]. — Режим доступа : <https://vuejs.org/guide/introduction.html>.
  - [22] Tailwind CSS Documentation [Электронный ресурс]. — Режим доступа : <https://tailwindcss.com/docs>.
  - [23] Axios Documentation [Электронный ресурс]. — Режим доступа : <https://axios-http.com/docs/intro>.
  - [24] Gradle User Manual [Электронный ресурс]. — Режим доступа : <https://docs.gradle.org/current/userguide/userguide.html>.
  - [25] PostgreSQL Documentation [Электронный ресурс]. — Режим доступа : <https://www.postgresql.org/docs/>.
  - [26] Visual Studio Code Documentation [Электронный ресурс]. — Режим доступа : <https://code.visualstudio.com/docs>.
  - [27] Myers, Glenford J. The Art of Software Testing / Glenford J. Myers, Corey Sandler, Tom Badgett. — 3 edition. — Wiley, 2011.
  - [28] Robbins, Jennifer Niederst. Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics / Jennifer Niederst Robbins. — 5 edition. — O'Reilly Media, 2018.